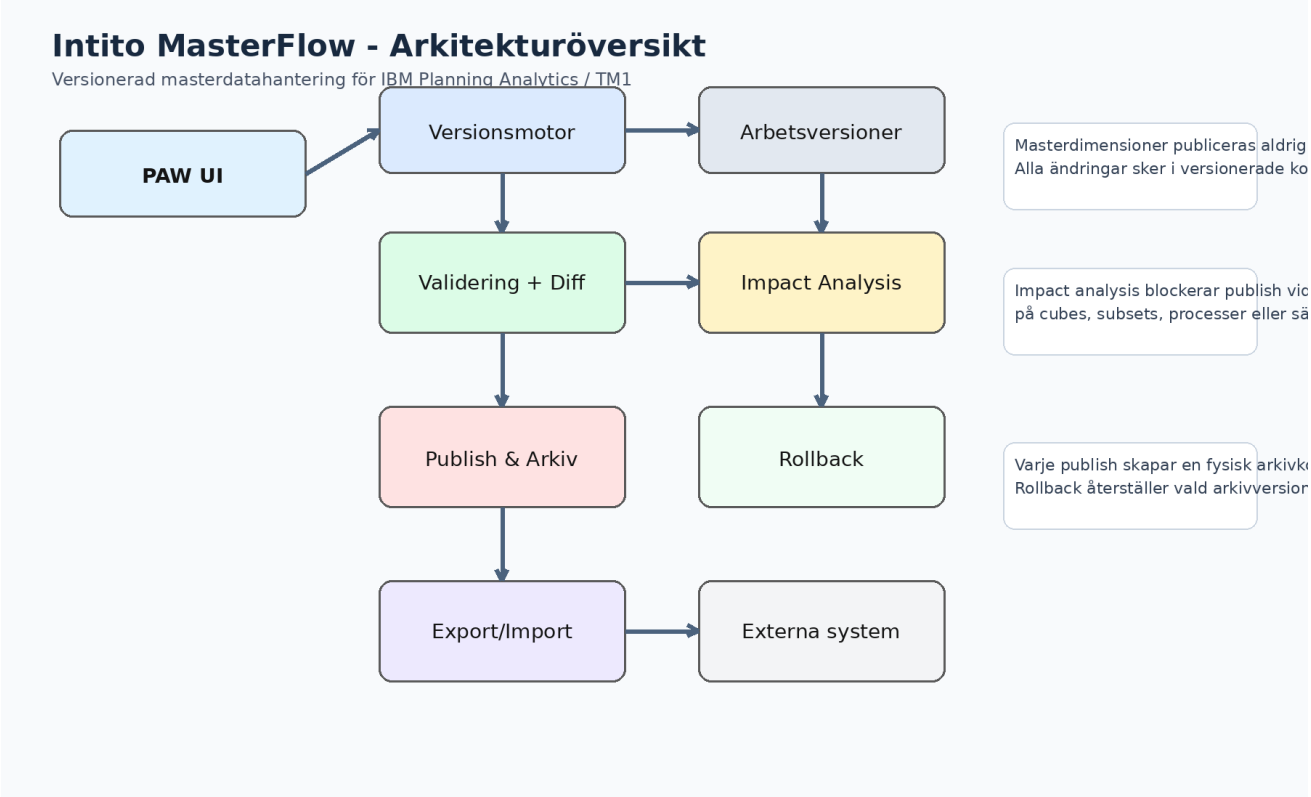
Intito MasterFlow

Fullständigt blueprint för ett MDM-ramverk i Planning Analytics / TM1

Version 1.0 • Blueprintpaket för implementation, drift och governance



Figur 1. Arkitekturöversikt för Intito MasterFlow.

Kärnförmågor	Teknik	Resultat
Versioner, impact analysis, publish, arkiv, rollback	PAW, TM1 TI, kontrollcubes, Git-pipeline	Kontrollerad masterdataförvaltning med full spårbarhet

1. Målbild och designprinciper

Intito MasterFlow realiserar masterdatahantering i TM1 genom att all förändring sker i separata, versionshanterade kopior av dimensioner. Masterdimensioner uppdateras endast via en kontrollerad publish-process.

Ramverket är generiskt och kan användas för flera dimensioner samtidigt. Det stödjer parallella arbetsversioner, attributhantering, export/import, arkivering av ersatta masterversioner och rollback till tidigare tillgängliga versioner.

Tre grundprinciper styr lösningen: master ändras aldrig direkt, impact analysis krävs före publish och alla förändringar måste vara spårbara i logg och historik.

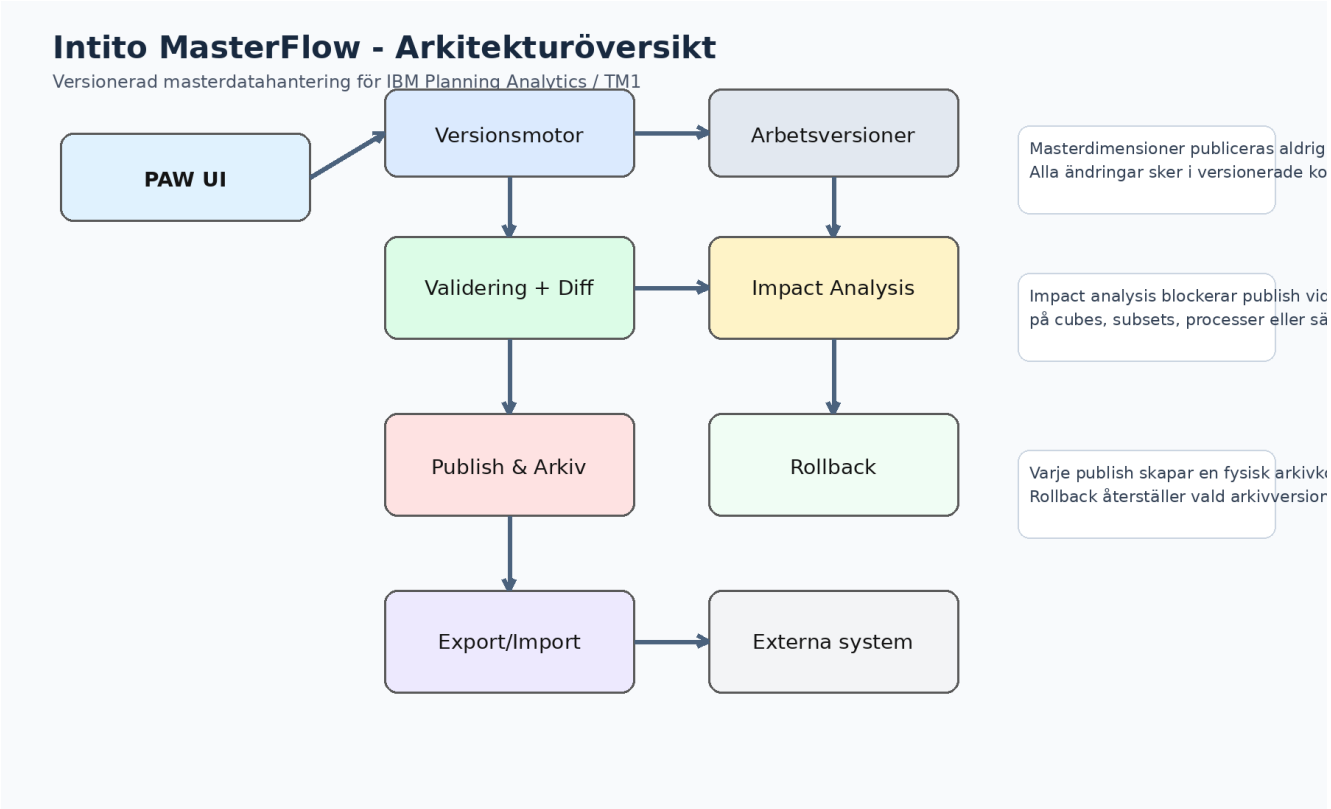
2. Fullständig datamodell

Datamodellen består av styrdimensioner, kontrollcubes, stagingobjekt och historikobjekt. Tabellen nedan sammanfattar kärnobjekten. Den detaljerade objektspecifikationen finns även som Excel-fil i paketet.

Objekt	Typ	Syfte	Kommentar
IMF.D.Dimension	Dimension	Lista över styrda dimensioner	En post per masterdimension
IMF.D.Version	Dimension	Versions-id	Möjliggör parallella arbetsversioner
IMF.C.Config	Cube	Konfiguration per dimension	Styr publish, impact, export/import och policies
IMF.C.Version	Cube	Metadata per version	Status, skapad av, publish-info, låsning
IMF.C.Diff	Cube	Skillnader mellan master och arbetsversion	Bas för impact analysis
IMF.C.Validation	Cube	Valideringsresultat	Tekniska och affärsmässiga blockerare
IMF.C.Impact	Cube	Påverkansanalys	Cube, subset, process, security, integration
IMF.C.ChangeLog	Cube	Detaljerad ändringslogg	Vem gjorde vad och när
IMF.C.PublishHistory	Cube	Historik över publish och rollback	Arkivnamn och återställningsspår
IMF.C.Stage.ParentChild	Staging	Canonical integrationsformat	Används för import/export

3. Arkitektur och processflöde

Arkitekturen delas i fem lager: PAW-gränssnitt, versionsmotor, validering/diff, impact analysis/publicering samt integration. Arbetsflödet är create version → edit → validate → impact → publish → archive → rollback vid behov.



Figur 2. Från PAW och versionsmotor till publish, arkiv och integration.

4. TI-processbibliotek

Nedan visas rekommenderad kärnuppsättning av processer. Processnamnen är avsedda att användas konsekvent i dokumentation, kod och test.

Kategori	Processer	Syfte
Version	IMF.P.Version.Create; CloneFromMaster; CloneFromVersion; Lock; Unlock	Skapa och hantera parallella arbetsversioner
Copy/Diff	IMF.P.Dimension.CopyAll; IMF.P.Compare.VersionToMaster	Kopiera struktur/attribut och skapa diff
Validering	IMF.P.Validate.Technical; Validate.Business	Tekniska regler och affärsregler före publish
Impact	IMF.P.Impact.RunAll; Impact.Cubes; Views; Subsets; Processes; Security; Integrations	Bedöma påverkan i modellen och nedströms
Publish	IMF.P.Commit.ArchiveMaster;	Arkivera aktiv master och

	Commit.PublishVersion	publicera version
Rollback	IMF.P.Rollback.Validate; Rollback.ToArchive	Återställ vald arkivversion på ett säkert sätt
Integration	IMF.P.Export.*; IMF.P.Import.*	CSV, SQL, JSON och HTTP i parent-child eller leaf-flat

5. Impact analysis-algoritm för TM1

Impact analysis använder diff som primär källa. För varje förändringsversion läses add, delete, move, relation, vikt, attribut och aliasändringar ut ur IMF.C.Diff.

Analysen går därefter igenom sex påverkansområden: cubes, subsets/views, processer, säkerhet, integrationer och affärspåverkan. Resultatet skrivs till IMF.C.Impact med severity-nivåerna Info, Warning, Error och Blocker.

Publish blockeras när BlockerCount > 0. Error får kräva manuell approval beroende på policy i IMF.C.Config.

Steg	Kontroll	Input	Output	Beslut
1	Läs diff	IMF.C.Diff	Ändringsmängd	Klassificera change type
2	Cube scan	Masterdimension + cube-metadata	Berörda cubes	Error vid delete som påverkar aktiv logik
3	Subset/View scan	Public subsets och views	Saknade/flyttade element	Warning eller Error
4	Process scan	TI-processdefinitioner	Kodberoenden	Error om antaganden bryts
5	Security scan	Dimension security och grupper	Påverkade securityobjekt	Blocker vid otillåten delete
6	Integration scan	Exportkontrakt och mappings	Påverkade payloads	Error vid leaf key/path brott
7	Business summary	Toppnoder, leaf counts, attribut	Sammanfattning för approver	Info/Warning beroende på omfattning

Rekommenderad pseudokod:

1) LoadDiff(versionId)

2) AnalyzeCubeImpact(diff)

3) AnalyzeSubsetImpact(diff)

4) AnalyzeProcessImpact(diff)

5) AnalyzeSecurityImpact(diff)

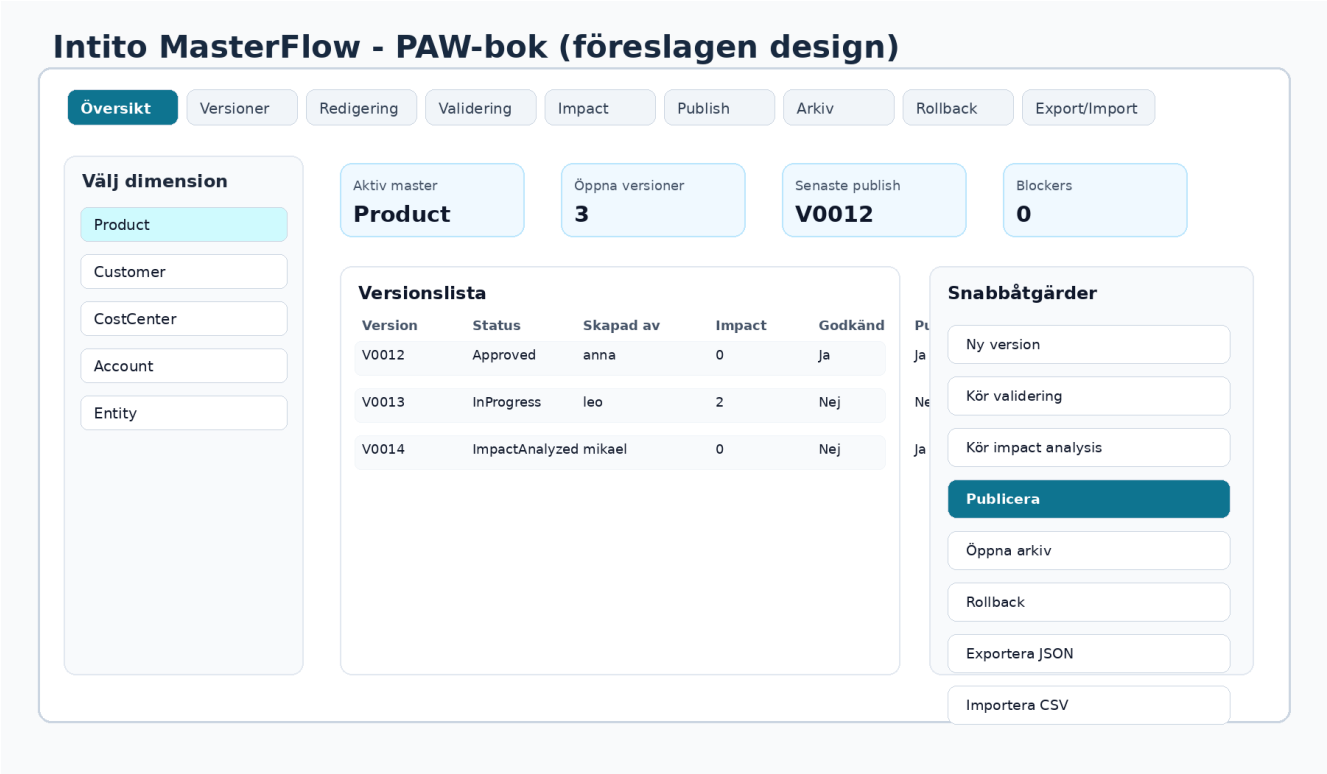
6) AnalyzeIntegrationImpact(diff)

7) SummarizeBusinessImpact(diff)

8) SaveImpactStatus(versionId)

6. PAW UI-design

PAW-boken ska fungera som styrgränssnitt för hela ramverket och samla versioner, validering, impact, publish, rollback och integration på en plats.



Figur 3. Föreslagen PAW-bok med översikt, versionslista och snabbåtgärder.

Flik	Funktion
Översikt	Aktiv master, öppna versioner, senaste publish och blockerare
Versioner	Skapa/klona/lås versioner, se status och ägare
Redigering	Öppna work-dimension i dimension editor och visa attribut
Validering	Tekniska fel, affärsregler och aliaskrockar
Impact	Cube-, subset-, process-, security- och integrationspåverkan
Publish	Förhandskontroll, arkivnamn och publicering
Arkiv	Historiska masterkopior och publiceringshistorik
Rollback	Analys av arkivversion och återställning
Export/Import	Val av format, shape och target/source

7. Git-baserad deployment pipeline

Blueprinten inkluderar en färdig GitHub Actions-pipeline som kan anpassas till valfritt deploymentsätt mot TM1. Pipelinen delar upp flödet i lint, paketering, deploy till DEV, smoke tests, approval och deploy till PROD.

Alla AI-genererade artefakter och all kod ska versionshanteras i Git. Deployment får endast ske från godkända branches och ska alltid kunna spåras till release-tag och commit.

Steg	Jobb	Mål	Kontroll
1	lint-docs	Validera markdown och YAML	Inga brutna länkar eller felaktiga filer
2	package-tm1	Paketera TM1-objekt	Release-tag matchar version
3	deploy-dev	Deploy till DEV	Smoke test för Version.Create och Compare
4	smoke-tests	Röktester i miljön	Publish ska blockeras på fel
5	approve-prod	Manuell approval	Impact analysis och rollback-plan granskad
6	deploy-prod	Deploy till PROD	Backup och spårbar release

8. AI-assisterad masterdata governance

AI ska användas som stöd till design, kodgenerering, testfall, mappings och sammanfattningar – inte som automatiskt publiceringsbeslut. Mänsklig approval är obligatorisk före publish till master.

Prompts, designbeslut och AI-genererade artefakter ska sparas i repo. Alla AI-förslag ska vara granskningsbara och kunna härledas till en version, en commit och en release.

Kontrollområde	Policy
Direktskrivning till master	Ej tillåtet. AI får endast påverka arbetsversioner via kod eller förslag.
Approval	All publish till master kräver mänsklig godkänning.
Spårbarhet	Prompts, svar, kodändringar och releaser ska

	versionshanteras.
Säkerhet	AI får inte kringgå roller, security eller blockerande impact analysis.
Förvaltning	AI-förslag på regler eller mappings ska reviewas innan drift.

9. Rekommenderad implementationsordning

1. Bygg core metadata och versionsregister.

2. Implementera versionering, copy och attributhantering.

3. Implementera diff, teknisk validering och changelog.

4. Implementera impact analysis med blockerande regler.

5. Implementera publish, arkiv och rollback.

6. Implementera export/import och deployment pipeline.

7. Bygg PAW-boken och AI-governance i förvaltningen.

Leverabler i paketet

- Blueprint-dokument i Word-format.
- Objektspecifikation i Excel-format.
- Arkitekturdiagram och PAW-wireframe som PNG.
- Markdown-dokument för blueprint, impact analysis, PAW UI och governance.
- GitHub Actions-pipeline i YAML-format.